

Ejercicio 02: ensambles, reducción dimensional y Green AI

INF-8239 Ciencia de Datos II, Unidad 01 (U01.E02, laboratorio U01.LAB03)

Estudiante	Roque Alexis Ventura Vallejo
Matrícula	100805873
Repositorio	github.com/RoqueVentura/UASD-Maestria-Ciencia-De-Datos , Ciencia de datos II/Unidad I/Ejercicio 02

1. Dataset y protocolo

Uso el mismo dataset, target y partición aprobados en el Ejercicio 01: HCV Data, del repositorio UCI (Lichtinghagen, Klawonn y Hoffmann, DOI 10.24432/C5D612, licencia CC BY 4.0). Son analíticas de sangre de donantes y de pacientes con hepatitis C. El notebook lo descarga con `download_csv` si no existe en `data/raw/dataset.csv`.

El protocolo quedó fijado antes de comparar modelos y no se cambió para favorecer a ninguno:

Target	<i>hepatopatía</i> : <i>si</i> = hepatitis, fibrosis o cirrosis; <i>no</i> = donante sano.
Exclusiones	Las 7 filas de donantes sospechosos (etiqueta incierta), la columna Unnamed: 0 (ordenada por diagnóstico, fuga) y Category (fuente del target).
Partición	80/20 estratificada con <code>random_state=42</code> : 486 filas de entrenamiento y 122 de prueba, de las cuales 15 son enfermos.
Preprocesamiento	El del Ejercicio 01 dentro de un <code>ColumnTransformer</code> : mediana y estandarización para las 11 variables numéricas, moda y one-hot para Sex. La matriz densa de entrenamiento tiene 486×13 y ocupa 49.4 KB.
Métrica	F1-macro como métrica principal. Clase prioritaria <i>si</i> : el error más costoso es el falso negativo, un enfermo clasificado como sano.

2. Modelos y medición

Comparé siete configuraciones con el mismo preprocesamiento: regresión logística como referencia (*logistic*), SVM RBF con $C=1$ y $C=10$ (*svm_c1*, *svm_c10*), Random Forest con 100 y 300 árboles y profundidad máxima 8 (*rf_100*, *rf_300*), HistGradientBoosting con profundidad máxima 6 (*boost*) y, como variante de reducción, la SVM con $C=1$ y PCA sobre el bloque numérico (*svm_pca*).

Cada modelo se ajustó tres veces y predijo tres veces sobre el conjunto de prueba, medidos con *perf_counter*, y se registra la mediana de cada medición. El pipeline se serializó con `joblib` en `reports/models` para medir su tamaño en disco. Todo se registra en `reports/green_ai_results.csv`, que también marca qué modelos forman la frontera de Pareto.

3. Resultados

Modelo	F1-macro	Recall <i>si</i>	FN	FP	Ajuste mediana (s)	Predicción mediana (ms)	Tamaño (KB)	Pareto
boost	0.960	0.867	2	0	0.087	3.6	181.6	Sí
rf_100	0.938	0.800	3	0	0.110	32.0	333.8	No
rf_300	0.938	0.800	3	0	0.266	31.3	984.2	No
logistic	0.914	0.733	4	0	0.009	2.3	4.5	Sí
svm_c10	0.902	0.800	3	2	0.016	3.2	17.2	No
svm_c1	0.844	0.600	6	1	0.015	3.3	18.2	No
svm_pca	0.844	0.600	6	1	0.016	3.2	19.3	No

Conjunto de prueba: 122 casos, 15 enfermos. FN es un enfermo clasificado como sano. La frontera de Pareto se calculó sobre F1-macro y mediana de ajuste para las siete configuraciones.

El boosting obtiene el mejor F1-macro y el menor número de falsos negativos: detecta 13 de los 15 enfermos, frente a 9 de la SVM base del Ejercicio 01. Ningún ensamble produce falsas alarmas. *rf_300* obtiene el mismo resultado que *rf_100*, pero cuesta unas 2.4 veces más tiempo de ajuste y casi 1 MB, así que queda dominado.

4. Reducción dimensional: PCA y t-SNE

El dataset tiene una variable categórica (Sex), así que el PCA se aplica solo al bloque numérico ya imputado y estandarizado, y Sex pasa por el one-hot sin reducir. Para conservar el 95 % de la varianza el PCA necesita 10 de las 11 variables numéricas, así que casi no reduce la dimensión. *svm_pca* obtiene exactamente el mismo resultado que *svm_c1* (F1-macro 0.844, 6 FN), pero tarda algo más en ajustarse (0.016 s frente a 0.015 s) y ocupa algo más. La Figura 1 muestra la varianza explicada acumulada. Coincide con lo visto en el Ejercicio 01: la varianza está repartida entre casi todas las variables y el PCA no aporta beneficio en este dataset.

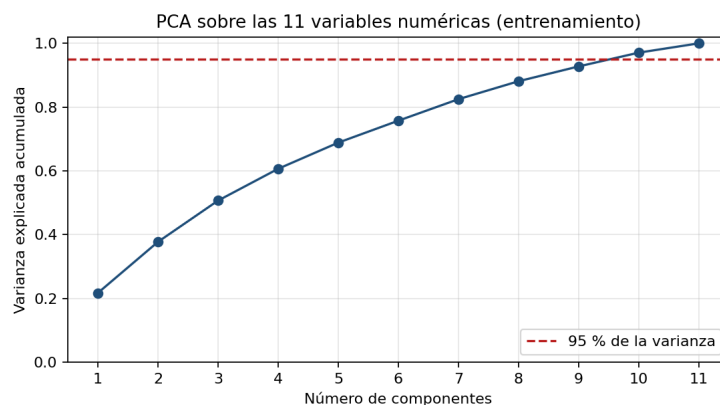


Figura 1. Varianza explicada acumulada del PCA sobre las 11 variables numéricas del conjunto de entrenamiento. La línea discontinua marca el 95 %.

Los mapas t-SNE usan las variables numéricas estandarizadas, perplexity 30 e `init="pca"`, con las semillas 42 y 7. Los dos son prácticamente iguales. En ambos, la mayoría de los enfermos forma pequeños grupos en la periferia y algunos quedan mezclados con los donantes, lo que es coherente con los falsos negativos de los modelos. Esa separación visual no valida por sí sola un clasificador.

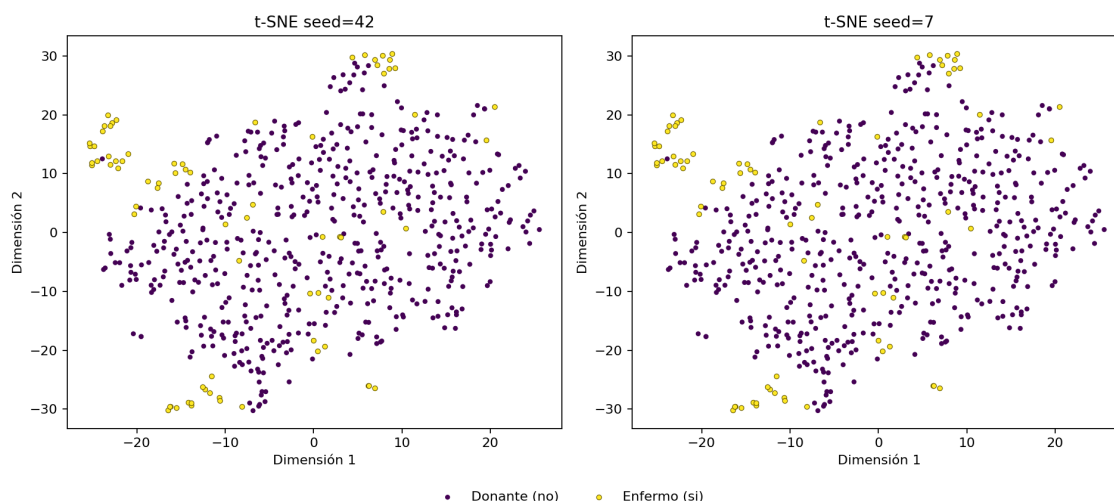


Figura 2. t-SNE con semillas 42 y 7. Morado: donante (*no*); amarillo: enfermo (*si*).

5. Frontera de Pareto y decisión

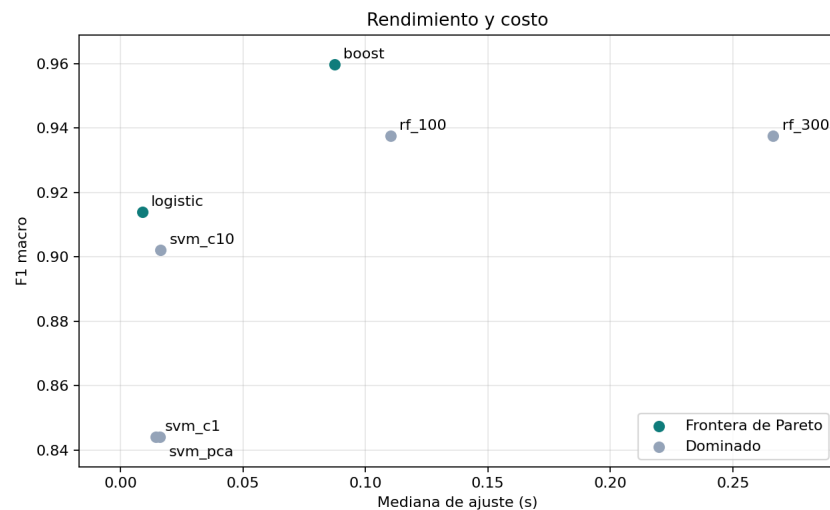


Figura 3. F1-macro frente a mediana de ajuste. En verde, los modelos de la frontera de Pareto.

La frontera queda formada por *logistic* y *boost*. *rf_100* queda dominado por *boost*, que tiene más F1 y se ajusta más rápido (0.087 s frente a 0.110 s). La tabla compara los dos modelos de la frontera:

Modelo	F1-macro	Diferencia de F1	Ahorro de tiempo de ajuste	Tamaño frente a boost	Enfermos detectados
boost	0.960			181.6 KB	13 de 15
logistic	0.914	0.046	89.7 %	177.1 KB menos	11 de 15

Selecciono boost. La regresión logística ahorra el 89.7 % del tiempo de ajuste y el 97.5 % del tamaño, pero en términos absolutos son unos 0.08 s por entrenamiento y 177 KB en un problema que no se reentrena de forma continua. A cambio deja escapar a dos enfermos más, que es justamente el error prioritario del protocolo. La logística sería la opción razonable solo si el modelo tuviera que ejecutarse con recursos muy limitados.

6. Código, artefactos y pruebas

Carpeta o archivo	Contenido
notebooks/green_ai.ipynb	Protocolo, catálogo de modelos, medición, PCA, t-SNE, Pareto y decisión cuantificada. Se ejecuta completo con Run All sin errores.
src/inf8239_u01	<i>data.download_csv</i> descarga y valida el dataset; <i>green.pareto_flags</i> marca los modelos no dominados.
tests/test_green.py	Pruebas de la frontera de Pareto.
reports	<i>green_ai_results.csv</i> , <i>pca_varianza.png</i> , <i>tsne_two_seeds.png</i> , <i>pareto.png</i> , <i>conclusion.md</i> y los siete modelos serializados en <i>models/*.joblib</i> .
requirements.txt, README.md	Dependencias y README actualizado con el entorno, la instalación, el protocolo, la ejecución y la descripción de los resultados.

Las pruebas se ejecutan desde la raíz del proyecto con `python -m pytest -q` y `PYTHONPATH=src`:

Prueba	Qué comprueba
test_pareto_marks_dominated_rows	Con dos modelos de igual F1, marca como dominado el más lento y también el que es peor en ambos criterios.
test_single_model_is_pareto	Un único modelo siempre pertenece a la frontera.

Resultado: las 2 pruebas pasan (2 passed, pytest 9.1.1). Entorno de medición: Windows 11 Pro (compilación 26200), procesador Intel64 Family 6 Model 140, Python 3.10.11 y scikit-learn 1.7.2.

7. Conclusión

Mantuve el protocolo del Ejercicio 01 sin cambios y amplié la comparación con dos ensambles, una SVM menos regularizada y una variante con PCA. El boosting fue el mejor en esta partición: sube el F1-macro de 0.844 (SVM base) a 0.960 y reduce los falsos negativos de 6 a 2 sin añadir falsas alarmas. Los Random Forest también superan a la SVM, pero 300 árboles no aportan nada frente a 100 y multiplican el costo.

La reducción de dimensionalidad no ayudó. El PCA necesita casi todas las variables para conservar el 95 % de la varianza y deja el resultado igual con más tiempo de ajuste. El t-SNE es estable entre semillas y muestra que algunos enfermos se confunden con los donantes, lo que es coherente con los falsos negativos que persisten.

En la frontera de Pareto elegí el modelo de máximo F1 y no el más eficiente, porque el ahorro de la regresión logística es pequeño en términos absolutos y cuesta dos enfermos no detectados. Los tiempos dependen del equipo y cambian entre ejecuciones; no equivalen a consumo energético y no se midió energía ni CO2. Los Random Forest usan todos los núcleos, lo que afecta a su tiempo. Con 15 enfermos en prueba, cada acierto mueve el recall casi siete puntos, así que las diferencias entre modelos deben leerse con cautela. El trabajo es académico y no sustituye un diagnóstico médico.

8. Uso de inteligencia artificial

Herramienta	Claude Code (modelo Claude Opus 5.5, de Anthropic), usado desde Visual Studio Code en septiembre de 2026.
Tareas asistidas	Verificar que el notebook se ejecutara sin errores, calcular el recall, los falsos negativos y los falsos positivos de cada modelo a partir de los modelos serializados, añadir el gráfico de PCA y las leyendas de las figuras, ejecutar el notebook completo y redactar el borrador de este informe.
Decisiones propias	Mantuve el protocolo del Ejercicio 01, aprobé la nueva ejecución del notebook, ejecuté las pruebas, hice los commits y revisé el informe.
Verificación	Las cifras provienen de la ejecución del notebook guardada en el repositorio y coinciden con reports/conclusion.md y con la versión de reports/green_ai_results.csv guardada en el repositorio. Comprobé que las pruebas pasan.
Responsabilidad	Revisé el contenido final y asumo la responsabilidad de lo entregado.

9. Fuentes

Lichtinghagen, R., Klawonn, F., & Hoffmann, G. (2020). *HCV data* [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5D612>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9, 2579–2605.